

Revisão completa do backend Java

Projeto: Leilão de Cavalos. Este material explica o código do backend para revisão da prova: classes, objetos, encapsulamento, arrays, métodos, Scanner, exceções, API e regras de negócio.

1. Visão geral

O programa tem duas formas de uso: um menu de console e uma API HTTP local. Os dois usam o mesmo objeto `LeilaoService`, portanto as regras ficam centralizadas no backend.

```
main()
├─ carregarDadosIniciais()
├─ iniciarServidor()
└─ menu com Scanner
    └─ executar(opcao)
        └─ LeilaoService e classes model
```

Importante: os dados ficam em arrays na memória. Ao encerrar o Java, os dados cadastrados são perdidos.

2. Main.java

Campos principais

```
private static final Scanner scanner = new Scanner(System.in);
private static final LeilaoService service = new LeilaoService();
private static HttpServer servidor;
```

`static` significa que o campo pertence à classe. `final` impede trocar a referência. O `Scanner` lê o teclado e o serviço concentra os dados.

main

Carrega cavalos e participantes, inicia a API, mostra o menu e repete até a opção zero. O bloco `try/catch` impede que um erro de entrada encerre o programa.

Menu e switch

`exibirMenu` apenas imprime as opções. `executar` usa `switch` para chamar o método correspondente: listar, pesquisar, iniciar, dar lance, consultar histórico, encerrar, reabrir ou cadastrar.

Leitura

`lerInteiro`, `lerDouble` e `lerTexto` transformam a entrada do teclado e lançam `IllegalArgumentException` quando o valor não é válido.

Servidor

`iniciarServidor` cria um `HttpServer` em `127.0.0.1:8080`. `atenderApi` verifica método HTTP e caminho, chama o serviço e devolve JSON.

3. Cavalo.java

Representa o animal. Seus atributos são privados, demonstrando encapsulamento.

```
private final int id;
private String nome;
private String raca;
private int idade;
private String sexo;
private double valorInicial;
private boolean vendido;
```

O construtor valida ID, textos, idade e valor. Os getters permitem leitura controlada. `setVendido` altera somente o estado de venda. O método `toString` sobrescreve o método herdado de `Object`.

4. Participante.java

Representa quem dá lances. O construtor valida nome e CPF. `cpfValido` verifica formato, números repetidos e os dois dígitos verificadores usando laços e soma ponderada.

```
if (!cpfValido(cpf)) {
    throw new IllegalArgumentException("CPF inválido.");
}
```

```
}
```

O CPF é armazenado apenas com números. Isso facilita a comparação de duplicidade no serviço.

5. Lance.java

É um objeto simples que associa um participante a um valor e à data/hora do lance.

```
private final Participante participante;  
private final double valor;  
private final LocalDateTime data;
```

Os campos são `final` : não mudam depois da criação. `toString` mostra o participante, o valor formatado e a data.

6. Leilao.java

É a classe que concentra as regras principais. Possui um cavalo, um array de até 100 lances, a quantidade ocupada e o estado encerrado.

adicionarLance

1. Rejeita leilão encerrado.
2. Rejeita lance nulo, participante nulo, valor não finito ou não positivo.
3. Verifica limite do array.
4. Verifica no máximo duas casas decimais.
5. Compara com o valor inicial ou com o último lance.
6. Armazena o lance e incrementa a quantidade.

```
double valorMinimo = quantidadeLances == 0  
    ? cavalo.getValorInicial()  
    : lances[quantidadeLances - 1].getValor();
```

encerrar e reabrir

`encerrarLeilao` fecha o leilão e marca o cavalo como vendido quando existe lance.

`reabrirLeilao` abre novamente, desfaz a marcação de vendido e mantém o array de histórico.

7. LeilaoService.java

É a camada de serviço. Ela usa arrays fixos:

```
private final Cavalo[] cavalos = new Cavalo[50];
private final Participante[] participantes = new Participante[100];
private final Leilao[] leiloes = new Leilao[50];
```

Cada cadastro verifica nulo, limite e ID/CPF duplicado. Os métodos `getCavalos`, `getParticipantes` e `getLeiloes` devolvem cópias dos arrays para proteger a referência interna.

Buscas

`buscarCavalo` pesquisa parte do nome sem diferenciar maiúsculas.
`buscarCavaloPorId`, `buscarParticipantePorId` e `buscarParticipantePorCpf` localizam registros específicos.

iniciarLeilao

Procura um leilão existente. Se não existir, busca o cavalo, cria um objeto `Leilao` e armazena no array. Isso evita dois leilões para o mesmo cavalo.

resetar

Coloca todas as posições dos arrays em `null`, zera os contadores e permite carregar os dados iniciais novamente.

8. Exceções personalizadas

Exceção	Quando aparece
<code>CavaloNaoEncontradoException</code>	ID ou leilão inexistente.
<code>LanceInvalidoException</code>	Valor inválido, participante inexistente ou lance insuficiente.
<code>LeilaoEncerradoException</code>	Tentativa de lance após encerramento.

Elas herdam de `Exception`, por isso precisam ser declaradas ou tratadas com `throws` e `catch`.

9. API do backend

Método	Rota	Função
GET	/api/cavalos	Lista cavalos.
GET	/api/participantes	Lista participantes.
POST	/api/participantes	Cadastra participante.
POST/GET	/api/leiloes/{id}	Inicia ou consulta leilão.
POST	/api/leiloes/{id}/lances	Registra lance.
POST	/api/leiloes/{id}/encerrar	Encerra.
POST	/api/leiloes/{id}/reabrir	Reabre.
POST	/api/resetar	Restaura os dados iniciais.

A API captura exceções e responde com código HTTP e mensagem JSON.

10. Perguntas prováveis

1. Por que os atributos são `private` ? Para aplicar encapsulamento.
2. Por que existem contadores junto aos arrays? Porque o array tem tamanho fixo e nem todas as posições estão ocupadas.
3. Onde está a regra que impede lance menor? Em `Leilao.adicionarLance` .
4. O que acontece ao reabrir? O estado muda para aberto, o vendido volta a falso e os lances permanecem.
5. Qual a diferença entre `throw` e `throws` ? `throw` lança uma exceção; `throws` declara que o método pode propagá-la.
6. Por que usar `equals` em String? Porque `==` compara referências, não conteúdo.

11. Comandos

```
$fontes = Get-ChildItem backend/src -Recurse -Filter *.java | Select-Object -  
ExpandProperty FullName  
javac -encoding UTF-8 --release 17 -d backend/out $fontes  
java -cp backend/out Main
```

Checklist: saiba explicar cada classe, cada array, o fluxo do menu, o cadastro, a validação de CPF, a regra do lance, as exceções, as rotas da API e o método resetar.